DEPI | MICROSOFT DATA ENGINEER TRACK

CRM Sales Opportunities Data Warehouse

An end-to-end ELT project: dimensional modeling, dbt transformations, Airflow orchestration with Astronomer Cosmos, and DuckDB storage.

Document	Complete Project Documentation and Learning Guide
Dataset	Maven Analytics - CRM Sales Opportunities
Stack	DuckDB | dbt | Apache Airflow 3 | Astronomer Cosmos | Docker / Astro CLI
Author	Reem Sherif Mohamed Abd Elfatah
Version	3.0 - October 2026

How to read this document

It is written for a **reviewer** who wants to see what was built and proven, and for **you in the future** who wants to remember how and why. Each section explains the idea, then what was done, then the evidence. Before an interview, re-read Sections 7 (journey), 17 (issues), 18 (decisions) and 19 (lessons).

Contents

1. Executive Summary 2
2. Objectives, Scope and Method 3
3. Concepts Primer 3
4. Source Dataset 4
5. Technology Stack and Environment 4
6. Architecture 5
7. Project Journey: The 14 Checkpoints 5
8. Data Profiling and Data-Quality Findings 6
9. Dimensional Modeling Approach 6
10. Conceptual, Logical and Physical Schema 7
11. Implementation: Raw and Staging Layers 9
12. Implementation: Dimensions and Fact 10
13. dbt Project Configuration 11
14. Data-Quality Testing 11
15. Orchestration: Airflow, Cosmos and Docker 12
16. Validation, Results and Lineage 14
17. Issues Encountered and Resolutions 15
18. Decision Log 16
19. Lessons Learned and Debugging Playbook 17
20. How to Run 17
21. Project Structure and Final Packaging 18
22. Limitations and Future Work 18
23. Glossary 19
24. Appendix: Evidence Checklist 19

1. Executive Summary

This project builds a small but complete dimensional **Data Warehouse** for a B2B hardware company's sales pipeline. Four raw CRM CSV files are loaded into DuckDB, cleaned and modeled with dbt into a **star schema**, protected by **44 automated data tests**, and run end to end by an **Apache Airflow** DAG where **Astronomer Cosmos** shows every dbt model and test in the Airflow graph.

The business process is **sales opportunity management**. The fact table grain is **one row per sales opportunity**, so questions such as win rate by product series, revenue by regional office, or the performance of each sales manager are answered with simple joins.

Measure	Result
Fact rows	8,800 = source rows; 0 duplicates; 0 NULL or orphan foreign keys
Total close value	10,005,534 in raw = staging = fact (nothing lost in any join)
dbt data tests	44 / 44 pass. `dbt build` = 9 models + 44 tests = 53 nodes, PASS=53 WARN=0 ERROR=0
Validation script	12 / 12 reconciliation checks pass (Windows, container, and Windows again after Airflow)

Measure	Result
Airflow DAG	`load_raw_data` > `dbt_transformations` (19 nodes, 30 task instances) > `validate_dw`; Success in 2 min 53 s (DAG version 6)
Data-quality fixes	3 source defects (product name, sector spelling, office spelling) + 1 structural NULL pattern
Engineering issues solved	15 documented in Section 17, each with cause, fix and lesson

> **What makes this project strong**
>
> 1. **Proven, not claimed**: row counts, value totals and key counts reconcile from raw to fact.
> 2. **Every decision is explained** (Section 18), including the rejected alternatives.
> 3. **Failures are documented honestly**: a wrong prediction, a corrupted file, a database lock, a permission error.
> 4. **The tests were tested**: tests were broken on purpose to prove they can fail.

2. Objectives, Scope and Method

Assignment: build a dimensional model (conceptual, logical, physical) on the Maven Analytics CRM Sales Opportunities dataset, and an ETL/ELT pipeline with Airflow + dbt + DuckDB. Deliverables: code (.zip) and documentation.

- **Objectives:** three-level design; ELT where **dbt does every transformation and test**, DuckDB stores, **Airflow only orchestrates**; Cosmos integration; proof by reconciliation against the source; documented decisions and defects.
- **Out of scope:** incremental loads and slowly changing dimensions (static snapshot), scheduling (manual trigger), BI dashboards, cloud deployment.
- **Method:** numbered checkpoints (Section 7). Each was verified with real command output before the next one, and every decision came from measured data. On failure: identify the layer, read the real message, apply the smallest fix, re-run, record the lesson.

3. Concepts Primer

Term	Plain meaning	In this project
Fact table	Business events with numbers to measure.	`fact_opportunities`
Dimension	Descriptive table you slice facts by (who, what, when).	account, product, agent, date
Grain	What exactly one fact row means. Decide it first.	One sales opportunity
Star schema	One fact in the middle, denormalised dimensions around it.	1 fact + 4 dimensions
Surrogate key	A meaningless integer ID created by the warehouse.	`account_key`, `product_key`, ...
Degenerate dimension	An ID kept in the fact with no dimension table.	`opportunity_id`
Role-playing dimension	One dimension used several times with different meanings.	`dim_date`: engage and close
Unknown member	A special row (key -1) used instead of NULL.	Account “Unknown”, date “Not Available”
ELT	Load raw data first, transform inside the database (ETL transforms before loading).	Python loads raw; dbt transforms

Tool	What it is	Why it fits here
DuckDB	Embedded analytical database in one file.	Small analytical data, no server, portable.
dbt	Each transformation is a SQL SELECT file; dbt builds, orders, tests and documents them.	Versioned, testable SQL with automatic lineage. Owns all logic.

Tool	What it is	Why it fits here
Apache Airflow	Orchestrator: order, retries, status.	Required by the task. Holds no transformation SQL.
Astronomer Cosmos	Turns a dbt project into Airflow tasks.	Without it dbt is one opaque task.
Docker + Astro CLI	Runs Airflow in containers with one command.	Airflow is not a native Windows app; reproducible.

4. Source Dataset

File	Rows	Description	Columns
accounts.csv	85	Customer companies	account, sector, year_established, revenue, employees, office_location, subsidiary_of
products.csv	7	Products sold	product, series, sales_price
sales_teams.csv	35	Agents, managers, offices	sales_agent, manager, regional_office
sales_pipeline.csv	8,800	One row per opportunity	opportunity_id, sales_agent, product, account, deal_stage, engage_date, close_date, close_value
data_dictionary.csv	21	Column definitions	documentation only, not loaded

Stage	Rows	Data behaviour
Won	4,238	Closed successfully. Has account, dates and a close value.
Lost	2,473	Closed unsuccessfully. `close_value` is **0** (not NULL).
Engaging	1,589	Open. Has an engage date, no close date, `close_value` is NULL.
Prospecting	500	Open, not yet engaged. No engage date, no close date, `close_value` NULL.

Open deals = 1,589 + 500 = **2,089**, which equals the number of missing close dates: proof that the NULLs follow the business process and are not random. Date ranges: `engage_date` 2016-10-20 to 2017-12-27 (500 missing); `close_date` 2017-03-01 to 2017-12-31 (2,089 missing). The date dimension therefore covers 2016-01-01 to 2017-12-31 (731 days).

5. Technology Stack and Environment

Component	Role	Version
DuckDB	Analytical database (`warehouse/crm.duckdb`)	1.5.6
dbt-core / dbt-duckdb	Transformations, tests, documentation	1.12.5 / 1.11.0
Apache Airflow	Orchestration only	Airflow 3 (Astro Runtime 3.3-8)
Astronomer Cosmos	dbt models and tests as Airflow tasks	1.15.1
Astro CLI / Docker Desktop	Local Airflow in containers	1.46.0 / 29.8.1
Python	Scripts and virtual environment	3.11.9 local; 3.14.7 in container
Host	Windows, VS Code, PowerShell, WSL2 Ubuntu	15.6 GB RAM

Why Astro + Docker instead of WSL2: Cosmos and Astro are designed together; Airflow pins hundreds of packages, so dbt lives in its own virtual environment inside the container to avoid conflicts; and a reviewer can reproduce the setup with one command.

6. Architecture

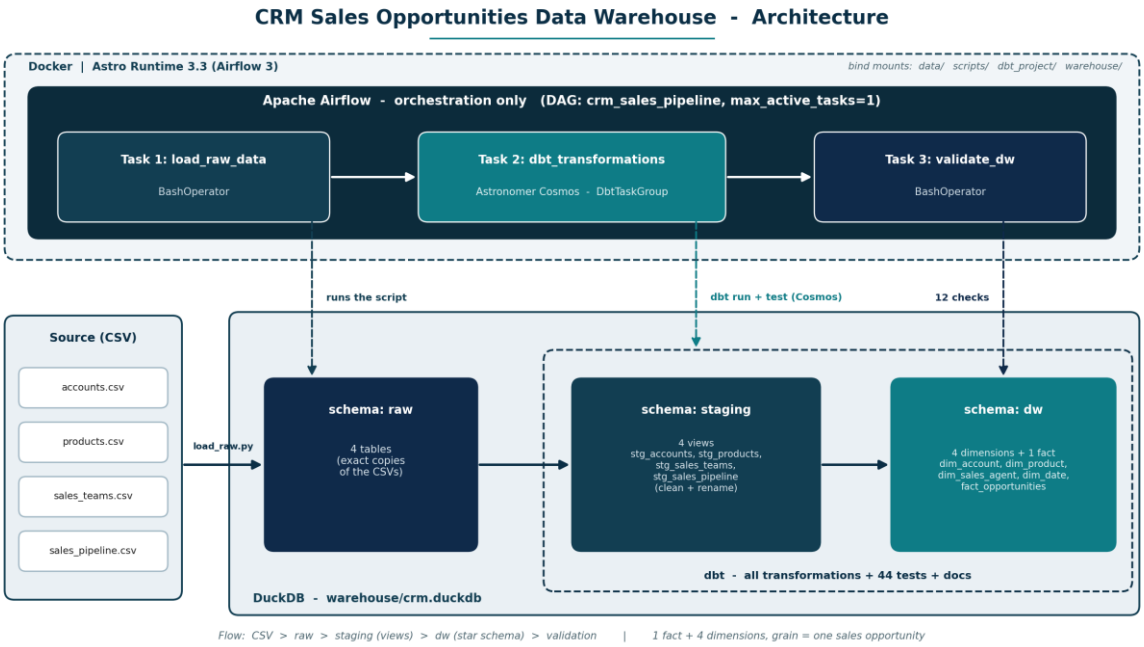


Figure 1 - End-to-end architecture

Layer	Schema	Built by	Content
Source	-	-	4 CSV files in `data/raw/`
Raw	`raw`	`scripts/load_raw.py`	Exact copies of the CSVs (4 tables). Never cleaned.
Staging	`staging`	dbt (views)	Renames, typo fixes. No joins, no business logic.
Warehouse	`dw`	dbt (tables)	4 dimensions and 1 fact table (star schema)
Validation	-	`scripts/validate_dw.py`	12 reconciliation checks against raw data

- **Raw stays a faithful copy.** All cleaning happens in staging, so every fix is visible, testable and reversible.
- **Dimensions and fact read from staging, never from raw.** A fix such as GTXPro is made in exactly one place.
- **No duplicated implementation.** dbt creates the `dw` tables; there is no separate DDL script.
- **Orchestration without logic.** The DAG only calls scripts and the dbt project.

7. Project Journey: The 14 Checkpoints

#	Checkpoint	What was done and what proved it
1	Load and verify data	Project folders; five CSVs present; approximate row counts checked.
2	Data profiling	Python/DuckDB scripts. Found **GTXPro**; NULLs tied to deal stage; Lost = 0, open = NULL.
3	Conceptual schema	Entities and relationships only (draw.io).
4	Logical schema	Star over snowflake; Unknown member (-1); grain fixed (dbdiagram.io).
5	Raw layer	Four tables under schema `raw`; counts 85 / 7 / 35 / 8,800.
6	dbt + DuckDB	dbt-duckdb installed; connection verified.
7	Staging models	4 views; fixed GTXPro, technolgy, Philipines; orphan products = 0.
8	Dimensions and fact	86 / 7 / 35 / 732 dimension rows; fact 8,800; Unknown counts 1,425 / 500 / 2,089; totals reconcile.
9	dbt tests	42 tests pass; break test proved they can fail; deprecation warnings fixed (12 places).
10	Airflow working	Astro CLI + Docker; test DAG succeeded.

#	Checkpoint	What was done and what proved it
11	Airflow + Cosmos + dbt	Bind mounts, isolated dbt venv, env-var DB path, `dbt build` PASS=51 in container, DuckDB lock solved.
12	Full pipeline	`load_raw_data` > dbt > `validate_dw` green.
13	Documentation	Physical schema generated from the database; dbt docs and lineage; dim_date edge fixed.
13+	Reconciliation as dbt tests	Two singular tests (44 tests, `dbt build` PASS=53). Broken on purpose (FAIL 1) and restored. DAG run Success in 2:53.
14	Final ZIP	To do (Section 21): exclude `.venv`, `target/`, `logs/`; `.gitignore`; test from a clean copy.

8. Data Profiling and Data-Quality Findings

Profiling ran **before** any modeling decision: `profile_data.py` (columns, types, nulls, duplicates, min/max), `profile_relationships.py` (null patterns by stage, key matches, illogical dates), `check_products.py` and `check_categories.py`. Schema choices (keys, grain, tests) must come from measured facts, not assumptions.

#	Finding	Evidence	Treatment
1	Product name mismatch	`sales_pipeline.product` held `GTXPro` (1,480 rows, about 17%) while `products` held `GTX Pro`. It was also the only product with no opportunities. A plain join would silently drop 1,480 rows.	Explicit mapping in `stg_sales_pipeline`; raw untouched; a relationships test proves 0 orphans.
2	Misspelled sector	`technolgy` for 12 companies; no correct spelling beside it.	Mapped to `technology` in `stg_accounts`.
3	Misspelled office	`Philipines` (1 row).	Mapped to `Philippines`.
4	NULLs from the business process	Account missing in 1,425 rows; engage_date in 500 (Prospecting); close_date in 2,089 (open deals).	Unknown member (key -1) in `dim_account` and `dim_date`.
5	Open deals have no value	`close_value` NULL for Engaging/Prospecting, 0 for Lost.	Preserved; the fact never invents values.
6	Sales hierarchy small and clean	6 managers (6+6+5+6+6+6 = 35 agents), 3 offices, no variants.	Kept inside `dim_sales_agent`.

> **How GTXPro was found, and the honest limit**
>
> Relationship profiling showed 1,480 pipeline rows with no matching product. Distinct product counts were equal (7 and 7) and exactly one product had no opportunity, which suggested one product spelled two ways: a **hypothesis**. `check_products.py` confirmed `GTXPro` vs `GTX Pro`. The fix went into staging, and the claim was closed only when the post-fix orphan count was 0.
>
> The conclusion rests on name similarity and mutual absence, not on price (the pipeline has no price column). `series`, `deal_stage` and `regional_office` were checked for variants before any rule was written; only the three typos above were found.

9. Dimensional Modeling Approach

Business process: managing sales opportunities for a company that sells hardware to other companies. **Grain:** one row in `fact_opportunities` equals one sales opportunity, identified by `opportunity_id`.

Concept	Applied as
Star schema	One fact table surrounded by four denormalised dimensions.
Degenerate dimension	`opportunity_id` stored directly in the fact.
Role-playing dimension	`dim_date` used twice: engage date and close date.

Concept	Applied as
Unknown member	Key -1 in `dim_account` (“Unknown”) and `dim_date` (“Not Available”) replaces NULL foreign keys.
Surrogate keys	`row_number() over (order by <natural key>)`; no extra dbt package.
Measure	`close_value`. Lost = 0 and open = NULL are kept distinct.

9.1 Decision: star versus snowflake

Manager and regional office stay **inside** `dim_sales_agent`. The dimension is tiny (35 agents, 6 managers, 3 offices), “performance per manager” needs one join instead of two or three, and the task asks for one sales-agent dimension. Snowflaking pays off for very large dimensions or when managers have many attributes of their own. A manager change would touch several rows, but the data is a static snapshot with no history.

9.2 Decision: optional relationships

Option	Pros	Cons
NULL foreign key	Simple.	Every INNER JOIN silently drops about 16% of deals; every query must remember LEFT JOIN.
Unknown member (chosen)	Joins work normally; no rows vanish; standard practice.	One fixed row per affected dimension and a COALESCE in the fact.

10. Conceptual, Logical and Physical Schema

10.1 Conceptual schema

Business story only: entities and relationships, no columns. Drawn in draw.io (`docs/conceptual_schema.png`).

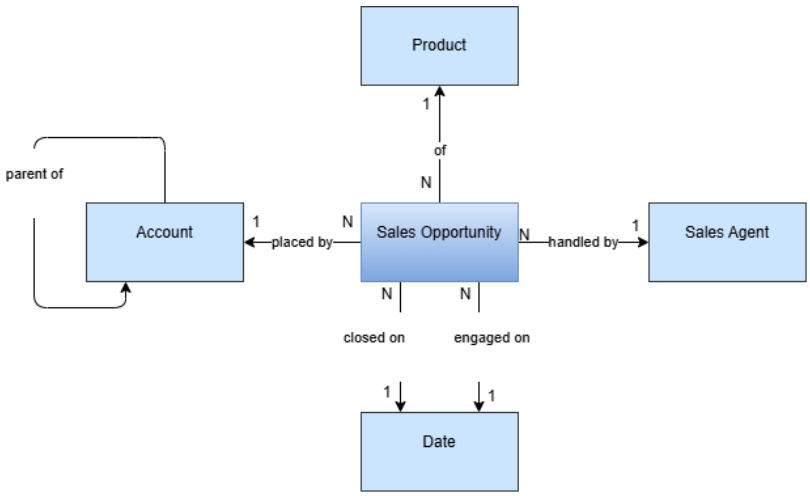


Figure 2 - Conceptual schema

- An **Account** places many **Opportunities**; an Account can be the **parent of** other accounts (from `subsidiary_of`).
- A **Sales Agent** handles many opportunities; a **Product** appears in many opportunities.
- Each opportunity has an engagement and a close date, both through the **Date** entity (role-playing).

10.2 Logical schema

Tables, columns, primary and foreign keys and grain, independent of any database. It is the contract the dbt models implement.

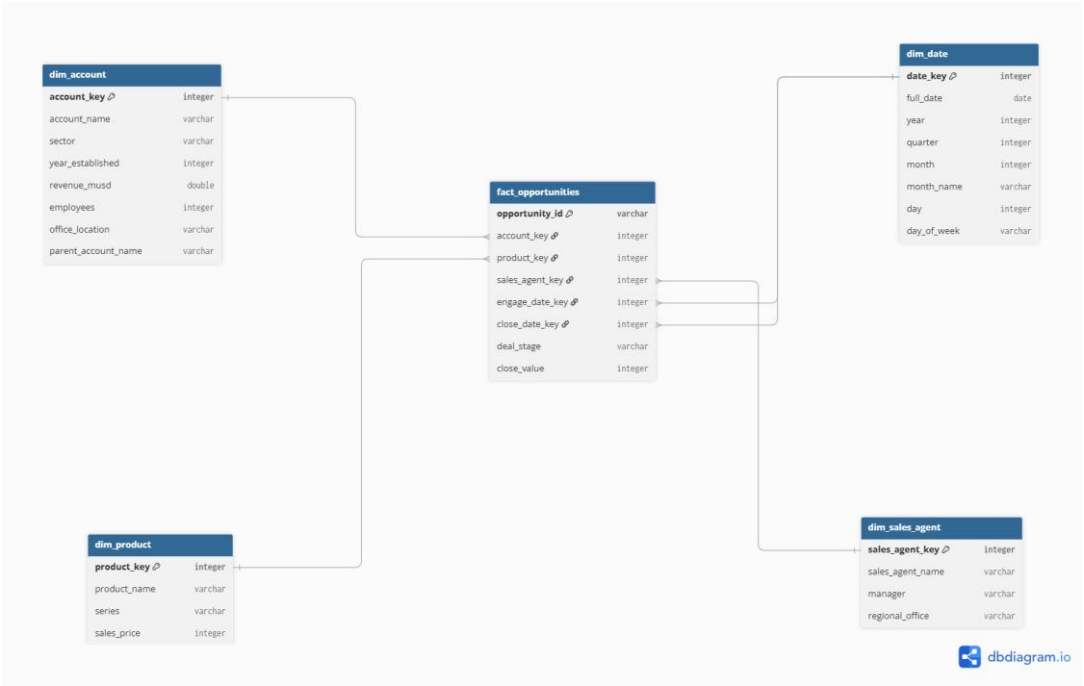


Figure 3 - Logical schema (star schema)

Table	Rows	Columns and notes
`fact_opportunities`	8,800	`opportunity_id` (pk, degenerate), `account_key`, `product_key`, `sales_agent_key`, `engage_date_key`, `close_date_key` (all fk), `deal_stage`, `close_value`
`dim_account`	86	`account_key` (pk), account_name, sector, year_established, revenue_musd (millions of USD), employees, office_location, parent_account_name. 85 accounts + Unknown (-1).
`dim_product`	7	`product_key` (pk), product_name, series, sales_price
`dim_sales_agent`	35	`sales_agent_key` (pk), sales_agent_name, manager, regional_office (denormalised)
`dim_date`	732	`date_key` (pk, YYYYMMDD), full_date, year, quarter, month, month_name, day, day_of_week. 731 days + Not Available (-1).

The Unknown member rule. About 16% of deals have no account or date yet because they are still open. A fixed row with key -1 is added to `dim_account` and `dim_date`, and the fact uses `COALESCE(key, -1)`. It was decided at Checkpoint 4, implemented at Checkpoint 8 (raw and staging stay untouched) and guarded by tests at Checkpoint 9.

Foreign key	Rows at -1	Reason
`account_key`	1,425	Open deals have no account yet
`engage_date_key`	500	Prospecting deals have not started engagement
`close_date_key`	2,089	All open deals have no close date
`product_key`, `sales_agent_key`	0	Always populated; a NULL would be a real defect, so these are **not** coalesced

10.3 Physical schema

What was **actually built** in DuckDB, generated from the live database by `scripts/describe_dw.py` (which also writes `docs/physical_schema.dbml`) instead of being copied from the design.

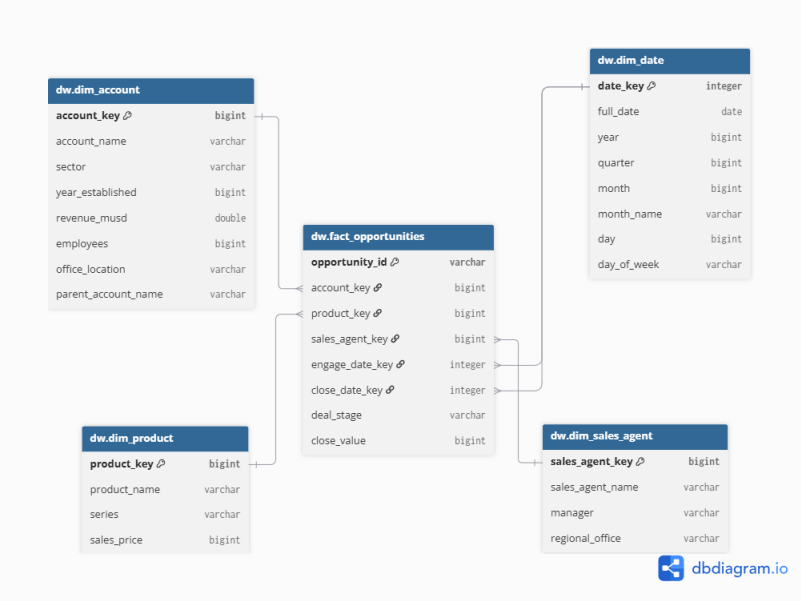


Figure 4 - Physical schema generated from the database

Schema	Objects	Materialisation
`raw`	4 tables: accounts, products, sales_teams, sales_pipeline	Tables (script)
`staging`	4 views: stg_accounts, stg_products, stg_sales_teams, stg_sales_pipeline	Views (dbt)
`dw`	5 tables: dim_account, dim_product, dim_sales_agent, dim_date, fact_opportunities	Tables (dbt)

Column(s)	Logical	Physical	Reason
Surrogate keys	integer	BIGINT	`row_number()` returns BIGINT in DuckDB
year, quarter, month, day (dim_date)	integer	BIGINT	DuckDB date functions return BIGINT
year_established, employees, sales_price, close_value	integer	BIGINT	`read_csv_auto` infers BIGINT
date_key, engage_date_key, close_date_key	integer	INTEGER	Matches; explicitly cast
revenue_musd	double	DOUBLE	Matches

> **Constraints are logical, not physical**
> dbt builds tables without PRIMARY KEY, FOREIGN KEY or NOT NULL constraints. The key markers in the diagrams are logical. Integrity is enforced by the 44 dbt tests and the validation script, a deliberate and common choice for analytical stores.

11. Implementation: Raw and Staging Layers

11.1 Raw layer

`scripts/load_raw.py` loads the four CSVs into schema `raw`. It is simple and idempotent.

```
ROOT    = Path(__file__).resolve().parent.parent
RAW_DIR = ROOT / "data" / "raw"
DB_PATH = Path(os.environ.get("CRM_DUCKDB_PATH", ROOT / "warehouse" / "crm.duckdb"))
CREATE SCHEMA IF NOT EXISTS raw;
CREATE OR REPLACE TABLE raw.<name> AS SELECT * FROM read_csv_auto('<file>');
```

- **Location independent**: paths come from `__file__`, so it runs from any folder, on Windows or in the container.
- **Same database path as dbt**: it reads `CRM_DUCKDB_PATH`, the variable dbt's `profiles.yml` uses.

- **Re-runnable**: `CREATE OR REPLACE` does not break existing staging views (required for repeated DAG runs; tested). Counts 85 / 7 / 35 / 8,800 on Windows and in the container.

11.2 Staging layer

One view per raw table: rename columns to the logical vocabulary and fix documented defects. No joins, no business logic. Sources are declared in `models/staging/sources.yml`.

Model	What it does
stg_accounts	Renames `account` to `account_name` and others; maps `technolgy` to `technology` and `Philipines` to `Philippines`.
stg_products	Standardises product column names.
stg_sales_teams	Standardises agent, manager and regional office names.
stg_sales_pipeline	Maps `GTXPro` to `GTX Pro`; standardises names. Types already correct, so no casts.

Each fix is an explicit mapping, not an automatic guess. **Verification:** 4 views; counts 85 / 7 / 35 / 8,800; 0 orphan products; technology = 12; Philippines = 1; old spellings = 0.

12. Implementation: Dimensions and Fact

Surrogate keys use `row_number() over (order by <natural key>)`. Ordering by a unique natural key keeps keys stable between runs while names stay unique (confirmed in profiling).

Dimension	Rows	Notes
dim_account	86	85 accounts + Unknown (-1) as a fixed row above the real rows.
dim_product	7	No Unknown member: product is never NULL in the pipeline.
dim_sales_agent	35	Keys 1..35 in alphabetical order; manager and regional office are columns.
dim_date	732	Generated date spine 2016-01-01 to 2017-12-31 + Not Available (-1). 2017-03-15 verified as a Wednesday in Quarter 1. No `full_date` test, because the Unknown row has a NULL date on purpose.

`fact_opportunities` has 8,800 rows and joins every dimension with a **LEFT JOIN**: an INNER JOIN silently removes unmatched rows, a LEFT JOIN keeps them and exposes a NULL key that a test can catch. `account_key`, `engage_date_key` and `close_date_key` are wrapped in `COALESCE(..., -1)` because their NULLs are expected. `product_key` and `sales_agent_key` are deliberately **not** coalesced: a NULL there is a real defect and must fail `not_null`.

Simplified illustration of the pattern (the real file is `models/marts/fact_opportunities.sql`):

```
select p.opportunity_id,
       coalesce(a.account_key, -1)  as account_key,
       pr.product_key,                       -- no coalesce on purpose
       ag.sales_agent_key,                   -- no coalesce on purpose
       coalesce(de.date_key, -1)    as engage_date_key,
       coalesce(dc.date_key, -1)    as close_date_key,
       p.deal_stage,
       p.close_value                         -- NULL open, 0 for Lost
from {{ ref('stg_sales_pipeline') }} p
left join {{ ref('dim_account') }} a ... left join {{ ref('dim_product') }} pr ...
left join {{ ref('dim_sales_agent') }} ag ... left join {{ ref('dim_date') }} de ... dc ...
```

Verification	Result
Fact rows	8,800, no duplicates; no NULL or orphan key
Unknown members	1,425 / 500 / 2,089, identical to the profiling NULL counts
Total close_value	10,005,534 in fact, staging and raw

Verification	Result
Lost / open deals	Lost close_value = 0; Engaging and Prospecting = NULL
Won deals by office	1,438 + 1,629 + 1,171 = 4,238 deals; revenue 3,568,647 + 3,346,293 + 3,090,594 = 10,005,534

Analytical payoff: Central has the most deals, yet West earns the highest revenue, so West has the higher average deal value. This is the kind of question the star schema answers with one join.

13. dbt Project Configuration

```
dbt_project/
  dbt_project.yml     name crm_dbt; staging = views, marts = tables
  profiles.yml        DuckDB connection; path from CRM_DUCKDB_PATH
  macros/generate_schema_name.sql
  models/staging/     sources.yml, schema.yml, stg_*.sql (4)
  models/marts/       schema.yml, dim_*.sql (4), fact_opportunities.sql
  tests/              assert_fact_rowcount_matches_staging.sql,
                      assert_fact_close_value_matches_staging.sql
```

- **Exact schema names:** a `generate_schema_name` macro makes dbt use `staging` and `dw` instead of default prefixes.
- **DB path from an environment variable** so the same project runs on Windows, in Docker and under Cosmos (which may run dbt from a temporary folder, where a relative path would break):

```
path: "{{ env_var('CRM_DUCKDB_PATH', '../warehouse/crm.duckdb') }}"
# Windows: default relative path.   Container: CRM_DUCKDB_PATH=/usr/local/airflow/warehouse/crm.duckdb
```

- **Materialisation:** views for staging (cheap, always current), tables for marts (stable reporting layer).
- **Current test syntax:** parameters nested under `arguments:`. **Explicit dependency:** `-- depends_on: {{ ref('dim_date') }}` in the fact, so lineage and run order include `dim_date` (issue 12).

14. Data-Quality Testing

Python checks from profiling were manual and one-off. As dbt tests they are part of the pipeline: if tomorrow's data breaks a rule, the run stops and says so.

Test type	Applied to
`unique`	Surrogate keys of the dimensions and `opportunity_id`
`not_null`	Keys and foreign keys (not `full_date` in dim_date)
`relationships`	Every fact foreign key to its dimension; staging pipeline to staging products (proves the GTXPro fix)
`accepted_values`	`deal_stage` (fact and staging) and categorical lists taken from the real data, which also guards against the typos returning
singular (2)	`tests/assert_fact_*`: the fact must match staging in row count and in total `close_value`

Total: 44 data tests = 42 generic (13 on the fact alone) + 2 singular. `dbt build` runs 9 models + 44 tests = 53 nodes: `PASS=53 WARN=0 ERROR=0`. Earlier milestones quote 42 tests and 51 nodes because the singular tests did not exist yet.

14.1 Proving the tests can fail

A passing test is not evidence that it can fail. `Lost` was removed from the `accepted_values` list of `deal_stage`: the test failed with **FAIL 1** (`ERROR=1`, others passed); restoring it returned `PASS=42`.

> **Lesson from a wrong prediction**
> The expected result had been predicted as FAIL 2473 (the number of Lost rows). The real result was **FAIL 1**, because `accepted_values` groups violating rows by value and counts the distinct invalid values, not rows. The row count (2,473) appears only in the compiled query (`n_records`) under `target/compiled/`. Predictions must be verified against real output.

Deprecation warning: dbt warned `MissingArgumentsPropertyInGenericTestDeprecation`. Test settings (`values`, `to`, `field`) now live under `arguments:`; all **12** places were fixed. Counts differed between runs (4 vs 12) only because of **partial parsing**; `--no-partial-parse` showed the full picture and then no warnings.

14.2 Reconciliation as singular tests

A **singular** test is one SQL file in `dbt_project/tests/` that returns the rows that are wrong: 0 rows = pass. (A generic test is a reusable rule attached to a column in YAML.) Here the rule spans two models, so a singular test fits.

```
-- tests/assert_fact_rowcount_matches_staging.sql
select f.n as fact_rows, s.n as staging_rows
from (select count(*) as n from {{ ref('fact_opportunities') }}) f
cross join (select count(*) as n from {{ ref('stg_sales_pipeline') }}) s
where f.n <> s.n
-- tests/assert_fact_close_value_matches_staging.sql   (same shape, using coalesce(sum(close_value), 0))
```

- `coalesce(..., 0)` is needed because `sum` ignores NULLs. Both tests use `ref()`, so they appear in the lineage connected to `fact_opportunities` and `stg_sales_pipeline`.
- They overlap with `validate_dw.py` on purpose: the tests compare the fact with **staging** inside dbt; the script compares the result with **raw** after the whole run.
- **Break test:** staging temporarily filtered with `deal_stage <> 'Lost'` (row count) and `deal_stage <> 'Won'` (value). Result: `FAIL 1` on both (`Got 1 result, configured to fail if != 0`); after restoring, `PASS=2`, then `dbt build` PASS=53 and `validate_dw.py` 12 PASS.
- The value test was broken with **Won**, not Lost: Lost deals are 0, so filtering them out would not change the total and the test would wrongly pass. A break test only proves something if it changes what the test measures.

15. Orchestration: Airflow, Cosmos and Docker

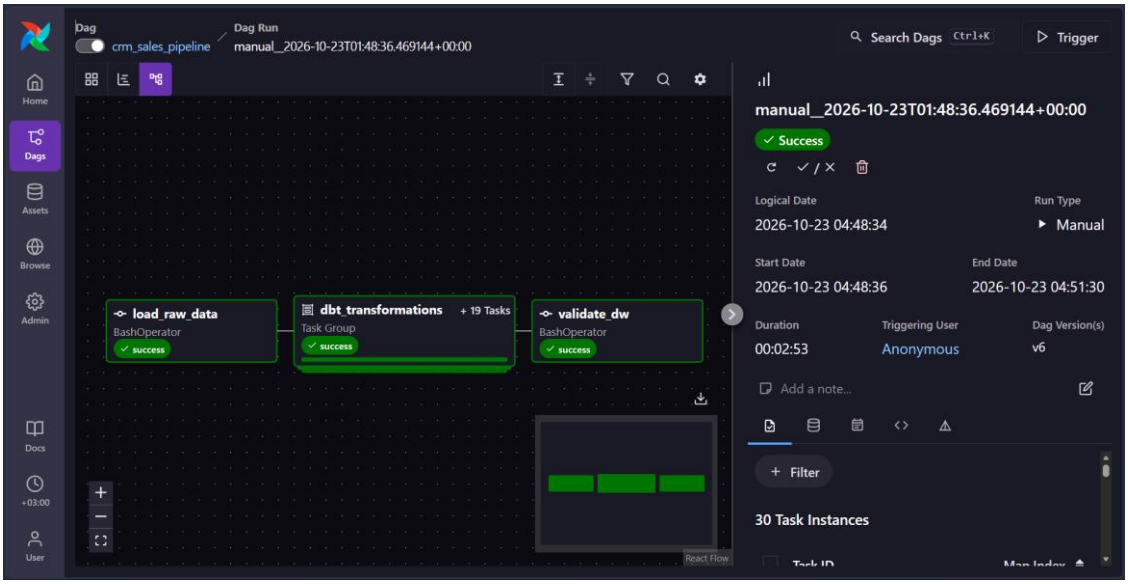


Figure 5 - DAG crm_sales_pipeline, successful run (DAG version 6, 30 task instances, 2:53)

Task	Operator	Purpose
`load_raw_data`	BashOperator	Runs `scripts/load_raw.py` with the dbt virtual environment's Python

Task	Operator	Purpose
`dbt_transformations`	Cosmos DbtTaskGroup	19 top-level nodes ordered by `ref()`: 9 model groups (`run` + `test`) and 10 standalone tests (8 `relationships` + 2 reconciliation)
`validate_dw`	BashOperator	Runs `scripts/validate_dw.py`; exits with code 1 on any failed check, failing the task

BashOperator is used for the two scripts because Cosmos is dedicated to dbt; a plain Python script is best called by the simplest operator. Airflow contains no transformation SQL.



Figure 5b - dbt_transformations expanded (zoom in on the original screenshot)

> **Reading the numbers**
> The group header says “19 Tasks” (top-level nodes). The run page says **30 task instances**: 9 models x 2 = 18, plus 10 standalone tests, plus `load_raw_data` and `validate_dw`. Of the 44 dbt tests, 34 sit inside the models’ `test` tasks and 10 are standalone. Tests that reference two models (all `relationships` tests and the two reconciliation tests) are drawn as their own tasks.

15.1 Cosmos and the isolated dbt environment

Cosmos object	Meaning
`ProjectConfig`	Where the dbt project is (path inside the container).
`ProfileConfig`	Uses the project’s own `profiles.yml` (DB path from `CRM_DUCKDB_PATH`).
`ExecutionConfig`	Which dbt executable to use: the isolated `dbt_venv`, not Airflow’s environment.
`DbtTaskGroup`	A group of tasks in the DAG; our own tasks go before and after it.

```
# airflow/Dockerfile (after the FROM line)
RUN python -m venv /usr/local/airflow/dbt_venv && \
    /usr/local/airflow/dbt_venv/bin/pip install --no-cache-dir \
    dbt-core==1.12.5 dbt-duckdb==1.11.0 duckdb==1.5.6
```

Versions match the local machine so the database file stays compatible between Windows and Linux. Cosmos is installed in Airflow’s main environment through `requirements.txt`. **Python 3.14.7** in the container was a concern; it was tested instead of assumed (installation succeeded, `dbt build` PASS=51), so no Python change was needed.

15.2 Bind mounts

The Astro build context contains only `airflow/`; the dbt project, scripts, data and warehouse are its siblings. `docker-compose.override.yml` mounts them so there is one copy of each file and one `crm.duckdb`: `dbt_project/`, `scripts/`, `data/`, `warehouse/` on the host appear under `/usr/local/airflow/` in the container. The warehouse is mounted as a **folder**, because DuckDB writes companion files (such as the write-ahead log) beside the database. The

scheduler mounts all four; the dag-processor mounts the dbt project and warehouse, because Cosmos reads the dbt project while parsing the DAG.

- A `touch` test only proves a file can be created, not that DuckDB can lock and write through a mount, so a temporary DuckDB file was written and read inside the container (`[(7,), (42,)]`) and then opened from Windows with the same result; `crm.duckdb` was untouched.
- Then `dbt debug` (connection OK) and a full `dbt build` (PASS=51) ran in the container, and `check_fact.py` on Windows returned identical numbers. Limit: two processes at once were not tested, so never open `crm.duckdb` from Windows while Airflow writes.

15.3 The single-writer constraint

DuckDB allows one writer process at a time. On the first run, five independent tasks started together and each launched its own dbt process; four failed within the same second:

```
Could not set lock on file "/usr/local/airflow/warehouse/crm.duckdb":
Conflicting lock is held in /usr/local/bin/python3.14 (PID 55)
```

- The failed tasks were red; their dependants were orange (`upstream_failed`), only a consequence. dbt `threads: 1` did not help: the conflict was between several **processes**.
- **Fix:** `max_active_tasks=1`. Runs then succeeded in 1:59 (version 2, dbt only) and 1:54 (version 3). After the reconciliation tests were added (version 6) the run has 30 task instances and succeeds in **2:53**, longer because more tasks run one at a time (expected, not separately measured).

```
@dag(dag_id="crm_sales_pipeline", start_date=datetime(2026, 1, 1),
     schedule=None, catchup=False, max_active_tasks=1)
def crm_sales_pipeline():
    load_raw_data = BashOperator(task_id="load_raw_data",
        bash_command=f"{PYTHON} {SCRIPTS / 'load_raw.py'}")
    dbt_transformations = DbtTaskGroup(group_id="dbt_transformations",
        project_config=ProjectConfig(dbt_project_path=DBT_PROJECT_PATH),
        profile_config=profile_config,
        execution_config=ExecutionConfig(dbt_executable_path=DBT_EXECUTABLE))
    validate_dw = BashOperator(task_id="validate_dw",
        bash_command=f"{PYTHON} {SCRIPTS / 'validate_dw.py'}")
    load_raw_data >> dbt_transformations >> validate_dw
```

16. Validation, Results and Lineage

`scripts/validate_dw.py` runs 12 reconciliation checks against raw after every run and exits with code 1 if any fails, so Airflow marks the pipeline as failed.

Check family	What it proves
Row reconciliation	Fact row count equals raw `sales_pipeline` row count (8,800)
Uniqueness	No duplicate `opportunity_id` in the fact
Key completeness	No NULL foreign key in the fact
Value reconciliation	Sum of `close_value` equals the raw total (10,005,534)
Unknown members	Rows at -1 equal the NULL counts in raw (1,425 / 500 / 2,089)
Referential integrity	Every fact foreign key exists in its dimension

It ran on Windows, inside the container, and again on Windows after an Airflow run (12 PASS each time), proving that what Airflow built in Linux is correct and readable from Windows.

Final results	
Raw rows (accounts / products / sales teams / pipeline)	85 / 7 / 35 / 8,800
Dimension rows (account / product / agent / date)	86 / 7 / 35 / 732

Final results	
Fact rows	8,800 = raw 8,800
Total close value	10,005,534 = raw 10,005,534
dbt tests	44 passed, 0 warnings, 0 errors (53 nodes)
Validation checks	12 passed
Airflow DAG run	Success in 2:53 (30 task instances)

16.1 dbt documentation and lineage

`dbt docs generate` and `dbt docs serve --port 8085 --profiles-dir .` produce an interactive site with descriptions, tests and the lineage graph.

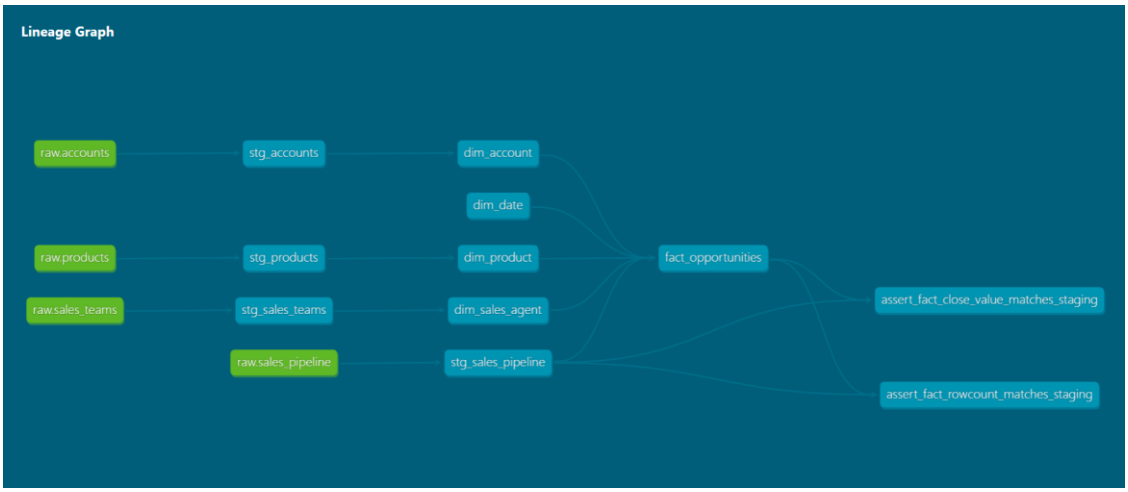


Figure 6 - dbt lineage graph (sources, staging, dimensions, fact, reconciliation tests)

- Lineage: `raw.*` > `stg_*` > `dim_*` > `fact_opportunities`, then the two reconciliation tests, each connected to `fact_opportunities` and `stg_sales_pipeline`.
- `dim_date` is generated, so it has no upstream node. It first had no edge to the fact either, because the fact joined it without `ref()`; dbt only draws lineage (and orders execution) from `ref()` and `source()`. Fixed in issue 12.

17. Issues Encountered and Resolutions

Every problem was found by checking real output, not by assumption.

#	Layer	What happened	Cause	Resolution	Lesson
1	Data	1,480 pipeline rows (about 17%) had no product	`GTXPro` vs `GTX Pro`	Mapping in staging; raw intact; relationships test	Silent join loss is the worst kind
2	Data	Sector `technolgy` (12), city `Philipines` (1)	Source typos	Explicit mappings in `stg_accounts`	Check categories before cleaning
3	Modeling	About 16% of deals would vanish from joins	NULL keys for open deals	Unknown member + LEFT JOIN + COALESCE	Model the business reality
4	Storage	"not a valid DuckDB database file"	Binary file opened in VS Code as text	Deleted and rebuilt with `load_raw.py`	Never open .duckdb in an editor
5	Execution	"No such file", "No dbt_project.yml found", "not an Astro project directory"	Commands run from the wrong folder	Scripts from root, dbt from `dbt_project\`, astro from `airflow\`; scripts made location independent	Know your working directory
6	Testing	Predicted FAIL 2473, actual FAIL 1	Wrong assumption about accepted_values	Corrected; test proven to fail and recover	Verify predictions
7	dbt	Deprecation warnings on generic tests	Old test-parameter syntax	Moved under `arguments:` in 12 places	Partial parsing hides warnings

#	Layer	What happened	Cause	Resolution	Lesson
8	Tooling	`astro` not recognised after install	Terminal kept the old PATH	Refreshed PATH or reopened VS Code	PATH is read at launch
9	Docker	Risk of port conflict on 8080 and 5432	Older Airflow project running	Stopped (not deleted) old containers	Stop, do not delete
10	Docker	Relative paths would break under Cosmos	Cosmos may run dbt from a temp folder	`CRM_DUCKDB_PATH` with default; `ROOT` from `__file__`	Avoid working-directory assumptions
11	Airflow	4 tasks failed: "Conflicting lock is held"	DuckDB single writer vs parallel tasks	`max_active_tasks=1`	Match concurrency to the database
12	dbt	No lineage edge from dim_date to fact	Fact joined dim_date without `ref()`	Added `depends_on` with `ref()`	Lineage comes from ref()
13	Docker	`dbt debug` reports git missing	No git in container; only needed for `dbt deps`	Not fixed on purpose (no `packages.yml`)	Do not add unneeded tools
14	Docker	"failed to execute cmd: exit status 1" on leaving the container shell	Last command (grep) returned 1, inherited by exit	Not an error; ignored	Read exit codes first
15	Windows	`mkdir tests` failed; `dbt test` failed with "Cannot open file crm.duckdb: Access is denied"; saving into `docs\` failed (EPERM)	Folders on D: give `Users`/`Everyone` read-only; full control only for `Administrators`/`SYSTEM`; shell not elevated. Why it appeared later: not determined	`icacls <folder> /grant "<PC>\<user>:(OI)(CI)M"` on `dbt_project` and `warehouse` (same remedy for `docs`), in a normal shell	"Access is denied" = permissions; "Conflicting lock" = another process

Harmless messages (do not chase these)

- "could not start proxy: proxy daemon is not supported on Windows": normal; Astro uses random ports, so use the address printed by `astro dev start`.
- The `db-migration` container shows `exited` (it runs once); 404 for favicon in the dbt docs; VS Code "Problems" count (spell checker); `KeyboardInterrupt` traceback after Ctrl+C on `dbt docs serve`.

18. Decision Log

Decision	Alternative	Why
Star schema, manager/office inside `dim_sales_agent`	Snowflake	Tiny dimension, simpler queries, matches the task
Unknown member (-1)	NULL foreign keys	No silent row loss; simple joins
Clean in staging, not in raw	Edit CSVs or raw tables	Raw stays a faithful copy; every fix visible and testable
dbt builds the `dw` tables	Separate CREATE TABLE scripts	One implementation, no duplication
Surrogate keys with `row_number()`	`dbt_utils` surrogate key	No extra package; stable because natural keys are unique
dbt in an isolated virtual environment	Install dbt with Airflow	Avoids conflicts; versions pinned to the local machine
BashOperator for scripts, Cosmos for dbt	One BashOperator running `dbt build`	Per-model visibility and retry
`max_active_tasks=1`	Parallel tasks	DuckDB single writer; correctness over speed
Environment variable for the DB path	Hard-coded relative path	Works from Windows, Docker and Cosmos temp folders
Constraints enforced by tests	PK/FK constraints in DuckDB	dbt tables carry none; tests are part of the pipeline
No git in the container	Install via `packages.txt`	Not needed; avoids an unnecessary tool

Decision	Alternative	Why
Reconciliation in dbt singular tests and in `validate_dw.py`	Only the script	Tests are in the lineage and stop the dbt run (fact vs staging); the script is an independent check against raw
Grant Modify on project folders	Run every terminal as Administrator	Files dbt creates stay owned by the normal user

19. Lessons Learned and Debugging Playbook

- **Profile before you model.** Every key choice traced back to a measured number.
- **Never fix data in raw.** Staging is where cleaning belongs.
- **Silent failures are the dangerous ones.** INNER JOINs and NULL keys lose rows without an error.
- **A test that has never failed is unproven.** Break it on purpose, and choose a break that changes what it measures.
- **Reconcile at two levels.** Inside dbt (fact vs staging, visible in lineage) and after the run (result vs raw).
- **Match the orchestrator to the database.** A single-writer database needs sequential tasks.
- **Avoid hidden location assumptions.** Relative paths work until a tool runs from somewhere else.
- **Tell permission errors from lock errors.** They need different fixes.
- **Do not add tools you do not need.** git in the container was correctly left out.

Step	Debugging playbook
1	**Which layer failed?** Data, dbt, DuckDB, Docker/WSL, Airflow or Cosmos. In Airflow, red is the cause; orange is only a consequence.
2	**Read the real message**, not the summary. Open the task log; the first red task has the root cause.
3	**Separate real errors from noise** (Section 17).
4	**Form a hypothesis, then test it** with a small command before changing anything.
5	**Apply the smallest fix**, then re-run the exact command that failed.
6	**Verify with numbers** (counts, totals), not just “no error”.
7	**Write the lesson down.** That is how Section 17 was built.

Interview-ready talking points

- What the grain is and why it is decided first; why Unknown members beat NULL foreign keys.
- How a one-letter spelling difference (`GTXPro`) would have dropped 17% of rows, and how it was found.
- dbt vs Airflow vs Cosmos responsibilities; why the DAG runs one task at a time (DuckDB single writer).
- Generic vs singular tests, and why the value break-test filters out Won, not Lost.

20. How to Run

Prerequisites: Python 3.11+, Docker Desktop, Astro CLI. PowerShell commands.

Without Airflow (from the project root)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/load_raw.py
cd dbt_project
dbt build --profiles-dir .            # 9 models + 44 tests = 53
cd ..
python scripts/validate_dw.py         # expect 12 PASS
```

Full pipeline with Airflow, and dbt docs

```
cd airflow
astro dev start                       # open the address it prints (random port on Windows)

# dbt documentation (from dbt_project)
dbt docs generate --profiles-dir .
dbt docs serve --port 8085 --profiles-dir .
```

Then trigger the DAG `crm_sales_pipeline` and wait for all tasks to turn green. Do not open `warehouse/crm.duckdb` from another tool while the DAG is writing to it.

Symptom	Likely cause and fix
"No dbt_project.yml found"	Run dbt from `dbt_project\`.
"not an Astro project directory"	Run astro from `airflow\`.
`astro` not recognised	Reopen VS Code completely so it reloads PATH.
Port already allocated	Stop other Airflow containers (`docker ps`, `docker stop`).
"Conflicting lock is held"	Another process has the DuckDB file open; close other tools; keep `max_active_tasks=1`.
"not a valid DuckDB database file"	Opened in an editor; delete and rebuild with `load_raw.py`.
"Access is denied" / EPERM	Windows permissions: `icacls <folder>`, then grant Modify to your own user on that folder only.

21. Project Structure and Final Packaging

```
CRM_Sales_Data_Engineering/
|-- data/raw/        accounts, products, sales_teams, sales_pipeline, data_dictionary (CSV)
|-- scripts/         load_raw.py, validate_dw.py, describe_dw.py, profile_*.py, check_*.py
|-- dbt_project/     dbt_project.yml, profiles.yml, macros/, models/, tests/
|-- airflow/         Dockerfile, requirements.txt, docker-compose.override.yml, .env, dags/
|-- warehouse/       crm.duckdb (generated)
|-- docs/            schemas, architecture, lineage, DAG images, physical_schema.dbml
`-- README.md
```

The diagnostic scripts (`profile_*`, `check_*`) are the evidence behind Section 8 and are kept as the profiling record.

Done	Checkpoint 14 checklist (final ZIP)
[ ]	Exclude `.venv`, `dbt_project/target`, `dbt_project/logs`, `.user.yml` and Airflow `logs`; add a matching `.gitignore`.
[ ]	Remove sample DAGs `hello_crm.py` and `exampledag.py` (screenshot already taken).
[ ]	Replace absolute `D:/...` paths in `docker-compose.override.yml` with relative paths and test them.
[ ]	Pin `astronomer-cosmos==1.15.1` in `requirements.txt` (currently unpinned).
[ ]	Unzip into a fresh folder and run Section 20 from scratch to prove it works on a clean copy.
[ ]	Remove the old `airflow_project_8b5b73` containers and images; add all figures and the PDF to `docs/`; finish `README.md`.
[ ]	The `icacls` changes only affect this machine; create the ZIP from a normal (non-Administrator) session.

22. Limitations and Future Work

- **Single writer:** DuckDB forces sequential tasks; a client-server warehouse would allow parallel builds.
- **Full refresh only:** incremental models would be needed for growing data. **No history:** no slowly changing dimensions.
- **Logical constraints only:** keys are enforced by tests, not by the database.

- **Manual trigger:** `schedule=None`. A production schedule, alerting and a BI layer on `dw` are natural next steps.
- **Source freshness and data contracts** (dbt) could detect upstream changes early.

23. Glossary

Term	Meaning
Bind mount	A host folder shown inside a container; edits appear on both sides.
DAG	Directed Acyclic Graph: the Airflow definition of tasks and their order.
Idempotent	Safe to run repeatedly with the same result (`CREATE OR REPLACE`).
Lineage	Dependency graph showing which table is built from which.
Materialisation	How dbt stores a model: view, table, and so on.
Orphan	A foreign key value with no matching dimension row.
Partial parsing	dbt re-reads only changed files to save time.
Reconciliation	Proving two datasets match by comparing counts and totals.
Singular test	A SQL file in `tests/` that returns failing rows; 0 rows = pass.
Task group	A named set of Airflow tasks shown together (Cosmos creates one for dbt).
Venv / ACL	An isolated Python environment / the Windows access-control list of a folder (`icacls`).

24. Appendix: Evidence Checklist

Item	Where	Shows
Architecture diagram	`docs/architecture.png`	Layers, tools and bind mounts (Figure 1)
Conceptual / logical / physical schema	`docs/*_schema.png`, `.dbml`	Figures 2 to 4
Airflow DAG	`docs/airflow_dag.png`	Successful run, 30 task instances (Figure 5)
dbt lineage	`docs/dbt_lineage.png`	Source-to-fact graph with 2 reconciliation tests (Figure 6)
dbt build result	terminal	PASS=53 (9 models + 44 tests) WARN=0 ERROR=0
Validation result	terminal	12 PASS
Add: break tests	terminal	accepted_values FAIL 1 then PASS=42; reconciliation FAIL 1 then PASS=2
Add: failed run (lock error)	Airflow UI + task log	Evidence for issue 11
Add: permission fix	terminal	`icacls` before and after (issue 15)